IEM

INSTITUTE OF ENGINEERING & MANAGEMENT

Salt Lake, Kolkata – 700091, India

A constituent institute of University of Engineering & Management, Kolkata, India

DATABASE MANAGEMENT SYSTEM / PROJECT REPORT

COLLEGE EVENT REGISTRATION AND RESOURCE BOOKING PORTAL

A relational database application for campus events, participation and shared resources

DEPARTMENT	ACADEMIC YEAR	SEMESTER	GROUP NUMBER
CSE(AIML)	2024–2028	3rd Year | 5th Semester	4

PROJECT TEAM

Submitted by

SL. NO.	NAME	ENROLLMENT NUMBER	ROLL NO.
1	AAYUSHMAN SINGH	12024052016080	81
2	GARV RAI	12024052016077	78
3	ROHIT JHA	12024052016055	55
4	SATYAM KUMAR	12024052016081	72

DEPARTMENT OF CSE(AIML)

Academic session 2024–2028 • Fifth semester • Group 4

Submitted as part of the Database Management System course

Abstract and report contents

ABSTRACT

Project at a glance

The College Event Registration and Resource Booking Portal is a full-stack web application for discovering college events and coordinating student participation. Students can browse and filter events, register while seats remain, or enter a first-in, first-out waitlist. Event pages support reservations for rooms and shared equipment. An event-specific QR code opens check-in, where attendance is recorded for a student with an active registration during the event. Attendance records can be viewed, and eligible attendees can download a PDF participation certificate.

The implementation uses a MySQL relational schema accessed by a Node.js/Express API and a browser interface written in HTML, CSS and vanilla JavaScript. The database separates students, events and resources from their relationship records, with primary and foreign keys, uniqueness rules, checks and indexes. Five automated request-validation tests pass; the supplied test suite does not automatically test the end-to-end application workflows.

REPORTING NOTE

Project details are drawn from the supplied source code, schema.sql, seed.sql, package.json and six supplied application screenshots. Institute and academic-session details follow the supplied E-Waste Collection System reference report.

CONTENTS

Sections and page numbers

SECTION	DESCRIPTION	PAGE
01	Cover and group details	1
02	Abstract and report contents	2
03	Introduction and problem statement	3
04	Objectives, scope and technology stack	4
05	System architecture and workflow	5
06	ER diagram	6
07–09	Database tables and column definitions	7–9
10	Relationships, integrity and normalization	10
11–12	DBMS implementation, SQL and testing	11–12
13–16	Application screenshots and captions	13–16
17	Conclusion and future scope	17
18	References	18

PROJECT CONTEXT

Introduction and problem statement

INTRODUCTION

Managing campus events through a shared portal

Colleges conduct academic, cultural and community events that involve participants, venues and shared equipment. A portal provides a central place to publish event details, discover activities, manage sign-ups and connect attendance to a student registration. This project implements those workflows as a database-backed web application.

Students can view event schedules, categories, locations and available capacity. Registration records connect students to events; registrations beyond event capacity are waitlisted. Organizers can define events and select room or equipment reservations. QR-based check-in records attendance for eligible participants, which in turn enables certificate issuance.

PROBLEM STATEMENT

Coordination and data consistency challenges

Scattered event information	Separate announcements and forms make event details and schedules harder to discover consistently.
Capacity and cancellation handling	Manual counting can result in inaccurate seat availability and inconsistent waitlist promotion.
Shared-space conflicts	Rooms and limited equipment need availability checks against overlapping event times.
Attendance verification	Organizers need a way to connect a check-in record to an eligible event registration.
Fragmented records	Student, event, resource and participation details should be linked and retrievable without duplicating core data.

PROJECT RESPONSE

The application centralizes event and resource records in MySQL and uses constraints and transaction-based workflows to protect key operations.

PROJECT GOALS

Objectives, scope and technology stack

OBJECTIVES

What the project is designed to accomplish

- Provide a browser-based view of upcoming events with category filters and schedule details.
- Record student registrations, enforce event capacity and place additional registrations on a FIFO waitlist.
- Allow event resource selections while checking quantities against overlapping reservations.
- Offer event-specific QR check-in for registered students while the event is in progress.
- Maintain attendance history and issue downloadable participation certificates after attendance.
- Demonstrate relational database design, constraints, indexes, parameterized queries and transactions.

SCOPE

Implemented behavior and stated boundaries

INCLUDED IN THIS PROJECT

Event discovery and filtering; publishing; capacity-aware registrations; waitlist promotion; resource booking; QR check-in; attendance records; certificate PDF generation.

CURRENT LIMITATION

The demo uses a student selector rather than production authentication. A login system and organizer authorization are not part of the current implementation.

TECHNOLOGY STACK

Languages, runtime, database and libraries

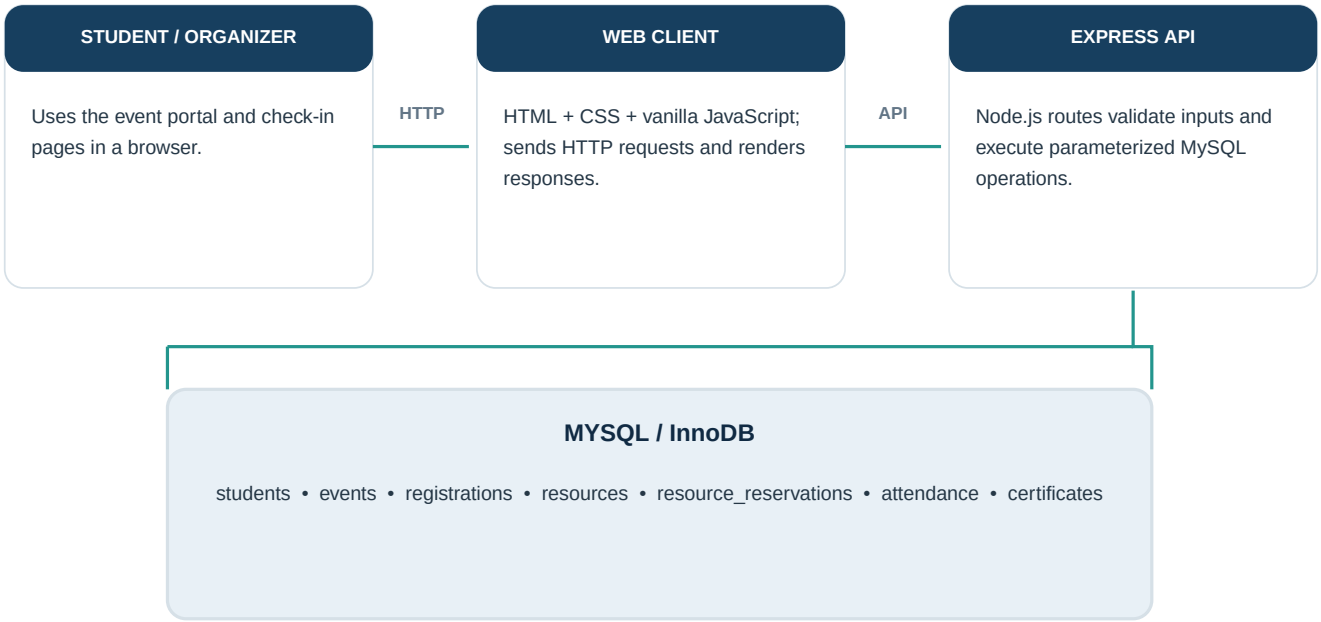
LAYER	TECHNOLOGY	ROLE IN THE PROJECT
Frontend	HTML, CSS, vanilla JavaScript	Structure, styling and browser interactions.
Backend	Node.js, Express	HTTP server and JSON API endpoints.
Database	MySQL 8+	Relational storage, constraints and InnoDB transactions.
Libraries	mysql2, qrcode, pdfkit	MySQL access, QR image generation and PDF generation.

The project manifest specifies Node.js 20 or newer. No hosted URL or source-control URL is claimed in this report.

System architecture and workflow

ARCHITECTURE

Browser, API and relational database



CORE WORKFLOW

Typical end-to-end event lifecycle

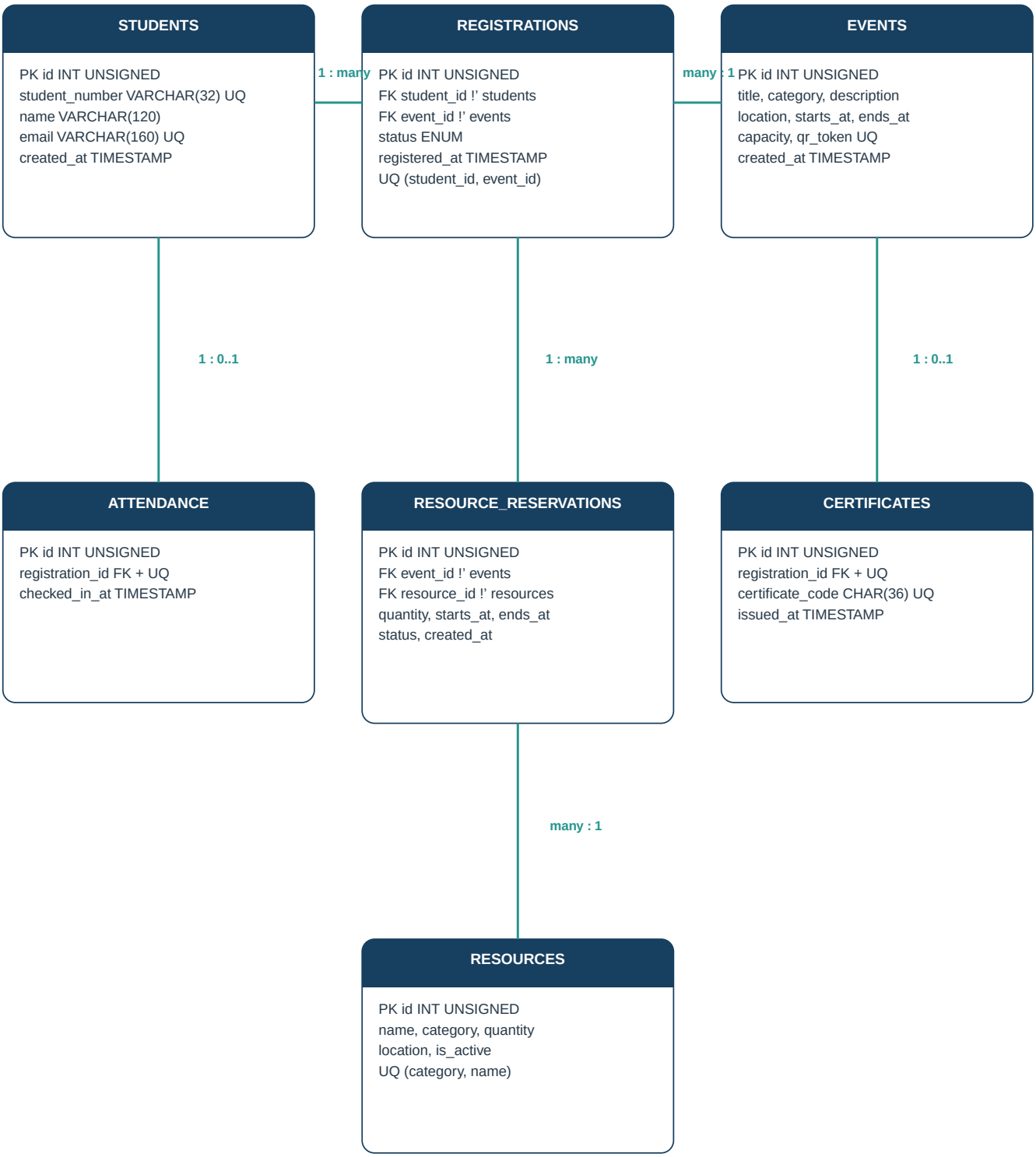
01	**Publish**	An event is created with schedule, capacity and a unique QR token.
02	**Discover & register**	Students browse events; a transaction locks the event while capacity is checked.
03	**Book resources**	Requested inventory is locked and checked against overlapping booked quantities.
04	**Check in**	An active registration is required; check-in is accepted only during the event interval.
05	**Review & certify**	Attendance is queryable; attended, registered students may receive a PDF certificate.

OPERATIONAL BOUNDARY

The student selector is a demonstration convenience, not identity verification or authorization.

Entity–relationship diagram

Generated from the seven CREATE TABLE definitions in the supplied database/schema.sql.



LEGEND

PK primary key • FK foreign key • UQ unique constraint • Relationship labels show parent-to-child cardinality.

SCHEMA REFERENCE

Table structures: students and events

Column names, MySQL types and constraints below match the supplied schema.sql.

TABLE 1

students

COLUMN	MYSQL TYPE	KEY / CONSTRAINT	PURPOSE
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique student identifier
student_number	VARCHAR(32)	NOT NULL, UNIQUE	Institution student number
name	VARCHAR(120)	NOT NULL	Student name
email	VARCHAR(160)	NOT NULL, UNIQUE	Unique email address
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Creation timestamp

REFERENTIAL ROLE

Referenced by registrations.student_id. Deleting a student cascades to associated registrations.

TABLE 2

events

COLUMN	MYSQL TYPE	KEY / CONSTRAINT	PURPOSE
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique event identifier
title	VARCHAR(180)	NOT NULL	Event title
category	VARCHAR(60)	NOT NULL	Filter/category label
description	TEXT	NOT NULL	Event description
location	VARCHAR(180)	NOT NULL	Event venue/location
starts_at	DATETIME	NOT NULL	Start of event interval
ends_at	DATETIME	NOT NULL	End of event interval
capacity	INT UNSIGNED	NOT NULL; CHECK > 0	Registration limit
qr_token	CHAR(36)	NOT NULL, UNIQUE	Event check-in token
created_at	TIMESTAMP	NOT NULL, DEFAULT	Creation timestamp

Additional event rules: chk_event_times enforces ends_at > starts_at; idx_events_schedule indexes (starts_at, ends_at).

Table structures: registrations and resources

TABLE 3

registrations

COLUMN	MYSQL TYPE	KEY / CONSTRAINT	PURPOSE
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Registration identifier
student_id	INT UNSIGNED	NOT NULL; FK !' students.id	Registered student
event_id	INT UNSIGNED	NOT NULL; FK !' events.id	Target event
status	ENUM('registered','waitlisted')	NOT NULL	Seat / queue state
registered_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Queue ordering time

REGISTRATION RULES

uq_registration_student_event prevents duplicate student/event pairs. Student and event foreign keys use ON DELETE CASCADE. idx_registration_waitlist covers (event_id, status, registered_at).

TABLE 4

resources

COLUMN	MYSQL TYPE	KEY / CONSTRAINT	PURPOSE
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Resource identifier
name	VARCHAR(140)	NOT NULL	Room/equipment name
category	ENUM('room','equipment')	NOT NULL	Resource type
quantity	INT UNSIGNED	NOT NULL, DEFAULT 1; CHECK > 0	Inventory units
location	VARCHAR(180)	NOT NULL	Storage / venue location
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Availability in portal

uq_resource_category_name enforces unique (category, name). idx_resources_category covers (category, is_active).

Table structures: reservations and records

TABLE 5

resource_reservations

COLUMN	MYSQL TYPE	KEY / CONSTRAINT	PURPOSE
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Reservation identifier
event_id	INT UNSIGNED	NOT NULL; FK !' events.id	Reserved-for event
resource_id	INT UNSIGNED	NOT NULL; FK !' resources.id	Reserved resource
quantity	INT UNSIGNED	NOT NULL, DEFAULT 1; CHECK > 0	Number of units
starts_at	DATETIME	NOT NULL	Reservation start
ends_at	DATETIME	NOT NULL; CHECK ends_at > starts_at	Reservation end
status	ENUM('booked','cancelled')	NOT NULL, DEFAULT 'booked'	Reservation state
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Creation timestamp

RESERVATION RULES

uq_event_resource enforces one row per event/resource pair. Event deletion cascades; resource deletion is restricted by its foreign key. Conflict index: (resource_id, status, starts_at, ends_at).

TABLE 6

attendance

COLUMN	MYSQL TYPE	KEY / CONSTRAINT	PURPOSE
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Attendance identifier
registration_id	INT UNSIGNED	NOT NULL, UNIQUE; FK !' registrations.id	Attended registration
checked_in_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Check-in time

TABLE 7

certificates

COLUMN	MYSQL TYPE	KEY / CONSTRAINT	PURPOSE
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Certificate record ID
registration_id	INT UNSIGNED	NOT NULL, UNIQUE; FK !' registrations.id	Eligible registration
certificate_code	CHAR(36)	NOT NULL, UNIQUE	Certificate identifier
issued_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Issue time

Relationships, integrity and normalization

RELATIONSHIPS

How the tables connect

RELATIONSHIP	CARDINALITY / DATABASE ENFORCEMENT
students !’ registrations	One student may have many registrations. Each registration references one student.
events !’ registrations	One event may have many registrations. Each registration references one event.
events !” students	Many-to-many, resolved by registrations; UNIQUE(student_id, event_id) allows at most one pair.
events/resources !’ resource_reservations	The junction table records each event/resource booking, quantity and time interval.
registrations !’ attendance	A registration may have zero or one attendance row; registration_id is UNIQUE.
registrations !’ certificates	A registration may have zero or one certificate row; registration_id is UNIQUE.

INTEGRITY

Keys, constraints, indexes and deletion behavior

- Primary keys uniquely identify rows; foreign keys prevent references to missing parent rows.
- UNIQUE constraints enforce distinct student numbers/emails, event QR tokens, student/event pairs, resource names by category, event/resource pairs, one attendance and one certificate per registration.
- CHECK constraints enforce positive event/resource/reservation quantities and end times later than start times.
- Indexes support event schedule ordering, waitlist selection and resource interval conflict checks.
- Deleting a student/event cascades through registrations; event deletion also cascades its resource reservations. Attendance and certificates cascade when their registration is deleted.

NORMALIZATION

Redundancy control in the relational design

The schema is structured in a normalization-oriented way: each entity has its own table; registrations and resource_reservations resolve many-to-many associations; and attendance/certificate facts are stored separately and reference a registration. Repeating groups are not modeled as packed values. This supports 1NF and a 3NF-style separation of entity facts from relationship facts. This report describes the design pattern, not a formal dependency proof for every business rule.

Transactions, locking and query behavior

TRANSACTIONAL REGISTRATION

Capacity check and FIFO waitlist

The registration route begins a MySQL transaction and locks the target event row with SELECT ... FOR UPDATE. It confirms the event exists, is not already started, checks the student, rejects duplicate registrations, counts rows with status = registered, then inserts either registered or waitlisted. The transaction commits the status decision together with the insert; errors cause rollback.

When an active registration is cancelled, the route locks the event and registration, deletes the selected registration, then selects the earliest waitlisted record ordered by registered_at and id with FOR UPDATE. If found, that row is updated to registered before the transaction commits. Ordering by id resolves ties at the same timestamp.

RESOURCE BOOKING

Inventory locking and interval-overlap checks

Resource replacement runs in a transaction. The event is locked; the currently booked and requested resource rows are locked in ascending ID order. The route checks active resources, room quantity rules and inventory bounds. It sums quantities for other booked reservations whose intervals overlap the event interval, rejecting a booking if existing quantity plus requested quantity exceeds the inventory. It then replaces the event's booked rows and commits.

INTERVAL OVERLAP

```
existing.starts_at < requested.ends_at  AND  existing.ends_at > requested.starts_at
```

CHECK-IN AND DATA ACCESS

Validation, joins and aggregate subqueries

- Request values are validated before use; SQL values are supplied using mysql2 execute placeholders.
- Event listing uses scalar COUNT subqueries to report active registrations and waitlist size.
- Attendance and certificate routes join registrations with students and events; check-in verifies the QR token, time window and registered status.
- A UNIQUE registration_id in attendance provides a database-level one-check-in-per-registration rule.

TRANSACTION SCOPE

These transaction and locking details are supported by the routes in src/app.js. The project does not include automated integration tests proving concurrent behavior.

Representative SQL and testing results

Illustrative excerpts reflect query patterns in src/app.js; placeholders are bound parameters.

1 / COUNT ACTIVE REGISTRATIONS

```
SELECT COUNT(*) AS total
FROM registrations
WHERE event_id = ? AND status = 'registered';
```

2 / FIFO WAITLIST PROMOTION

```
SELECT id, student_id
FROM registrations
WHERE event_id = ? AND status = 'waitlisted'
ORDER BY registered_at, id
LIMIT 1 FOR UPDATE;
```

3 / CHECK OVERLAPPING RESOURCE QUANTITY

```
SELECT COALESCE(SUM(quantity), 0) AS reserved
FROM resource_reservations
WHERE resource_id = ? AND status = 'booked' AND event_id <> ?
  AND starts_at < ? AND ends_at > ?;
```

4 / ATTENDANCE REPORT WITH JOINS

```
SELECT s.student_number, s.name, a.checked_in_at
FROM attendance a
JOIN registrations r ON r.id = a.registration_id
JOIN students s ON s.id = r.student_id
WHERE r.event_id = ?
ORDER BY a.checked_in_at DESC;
```

AUTOMATED TESTING

Request-validation test suite

TESTED INPUT	EXPECTED / ASSERTED BEHAVIOR	RESULT
Valid local date/time	Formats the valid value for MySQL.	PASS
Impossible / malformed dates	Throws HTTP 400 validation error.	PASS
Positive integer values	Accepts valid integer; rejects invalid cases.	PASS
Required text values	Trims valid text; rejects missing/blank values.	PASS
Request body shape	Accepts object; rejects absent, primitive or array.	PASS

Result: 5 automated tests passed with `npm test` (Node.js built-in test runner).

APPLICATION SCREENSHOTS

Event portal dashboard

Supplied application screenshot | Main portal, navigation and event overview

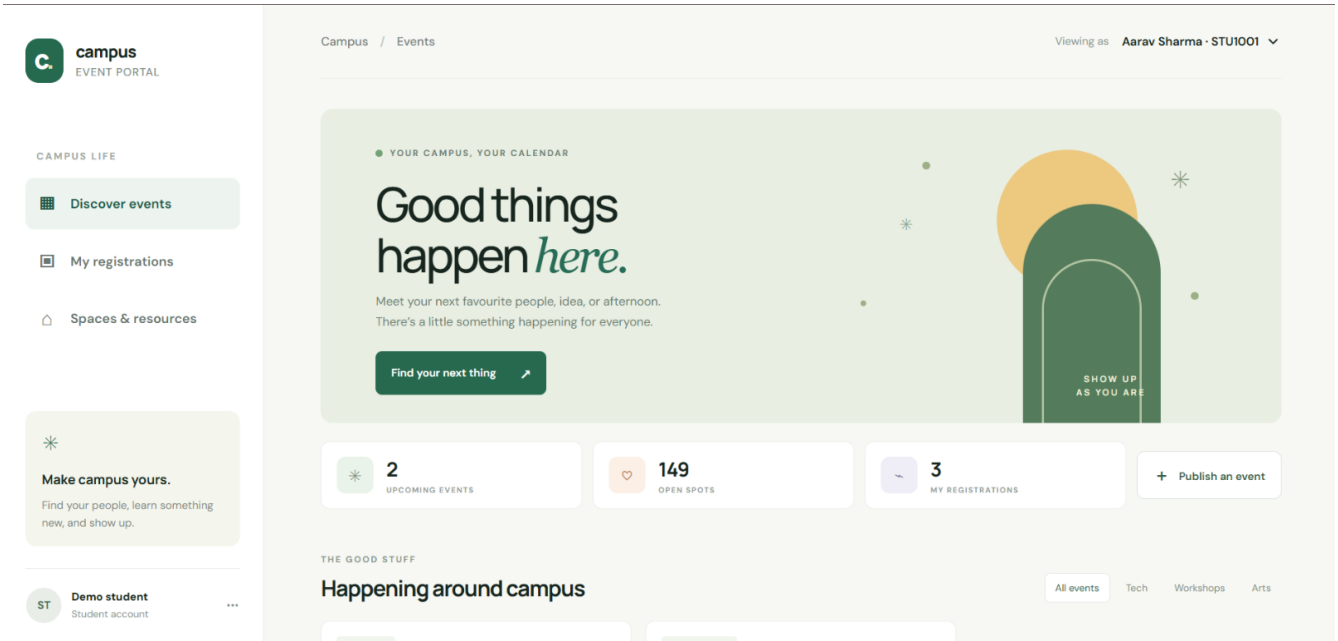


Figure 1. Campus Event Portal dashboard showing the main navigation, “Viewing as” student selector, campus introduction, event summary cards and event discovery area.

SCREENSHOT CONTEXT

The image is supplied project material. It illustrates the rendered interface; the counters and event content shown are screenshot state and should not be read as a permanent dataset.

VISIBLE FEATURES

What the dashboard demonstrates

- Navigation links for event discovery, registrations, and spaces/resources.
- Student selector, event overview metrics, and event publishing action.
- Category filters and event cards appear further down the page.

Event discovery and student plans

Supplied application screenshots | Event cards, filters and personal overview

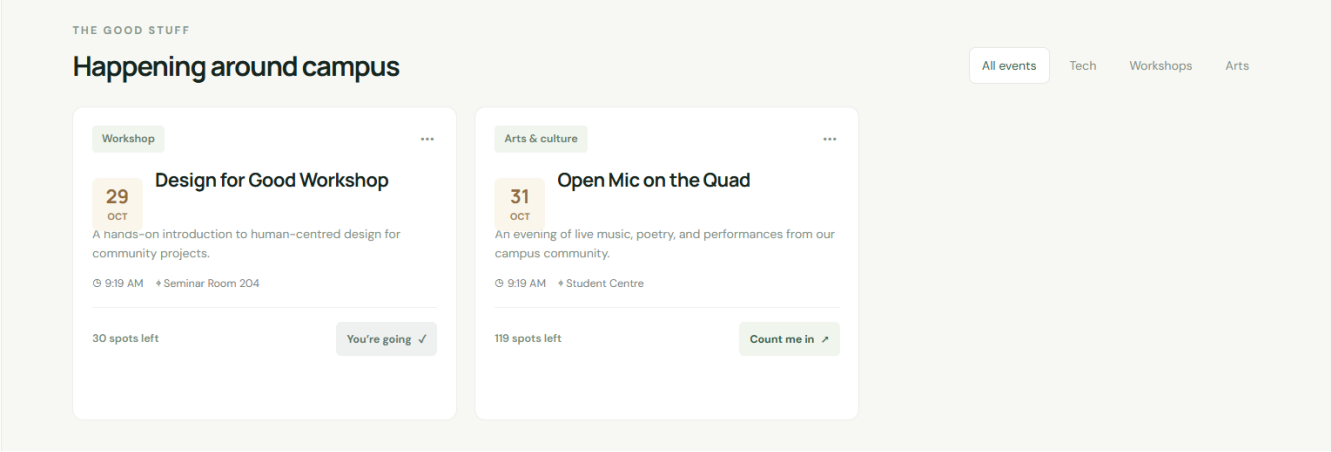


Figure 2. Event discovery area with category filters, event descriptions, locations, available spots and registration controls.

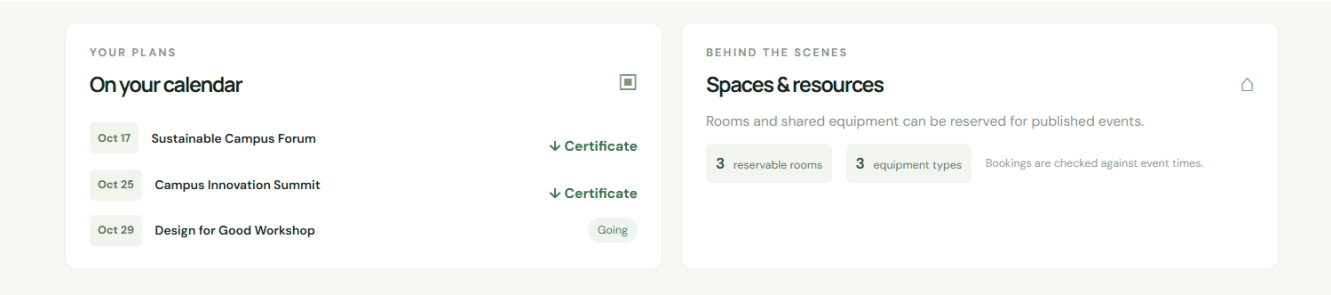


Figure 3. “On your calendar” registrations and the spaces-and-resources overview shown in the supplied application screenshot.

INTERPRETATION

Event dates, labels and counts are reproduced from the supplied screenshots. They are illustrative interface content and are not represented as a live system or current schedule.

Event details, QR and resources

Supplied application screenshot | Event detail dialog

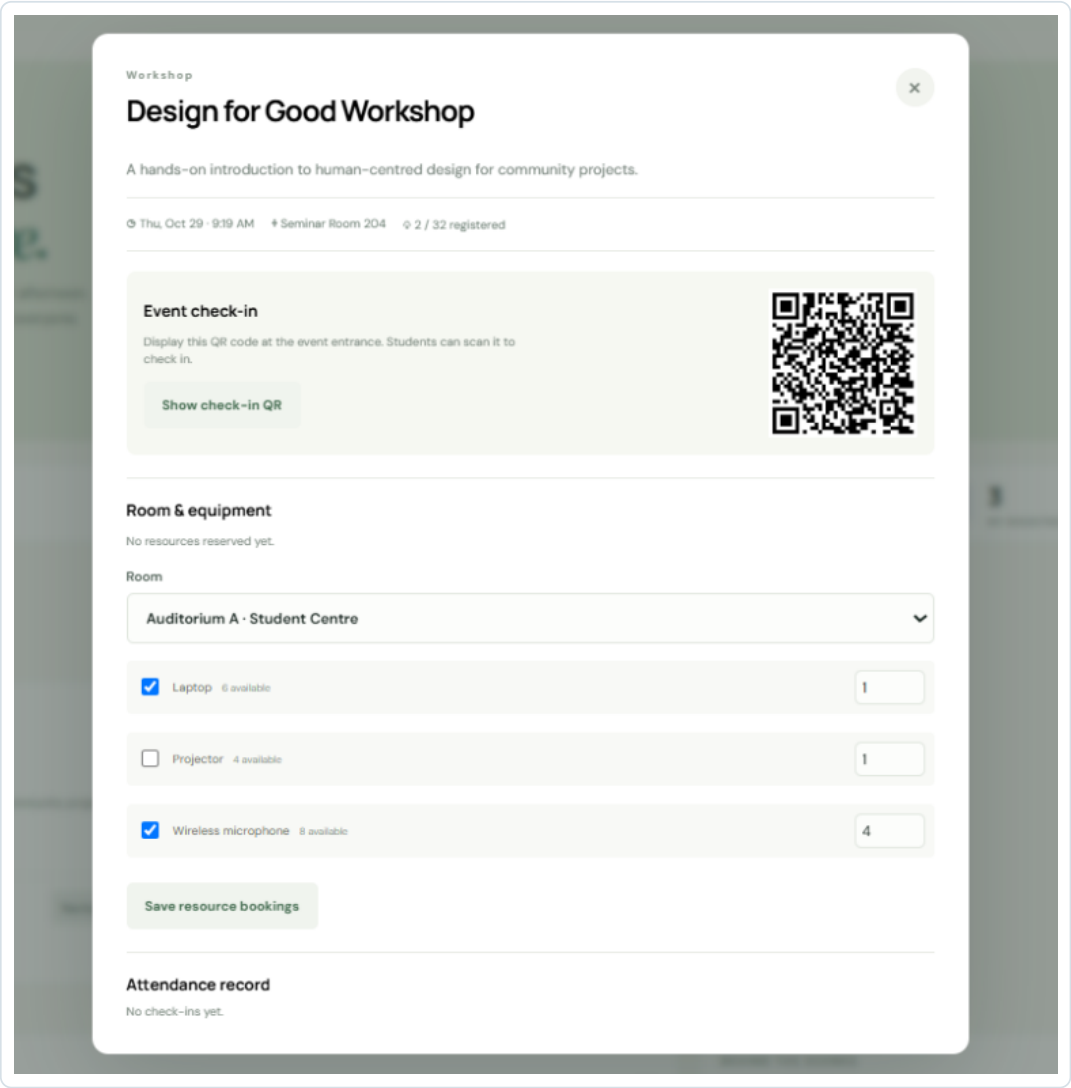


Figure 4. Event detail screen showing schedule and registration summary, an event QR-code panel, room/equipment reservation controls and the attendance-record area.

DATABASE FEATURES VISIBLE IN THIS SCREEN

The QR entry point corresponds to the event-specific check-in token workflow. Resource selections map to resource_reservations. The attendance panel is backed by attendance joined to registrations and students.

The screenshot shows an example interface state; it does not establish a successful live scan or reservation.

Event publishing and QR check-in

Supplied application screenshots | Event creation and check-in user interface

EVENT FORM

Publishing an event

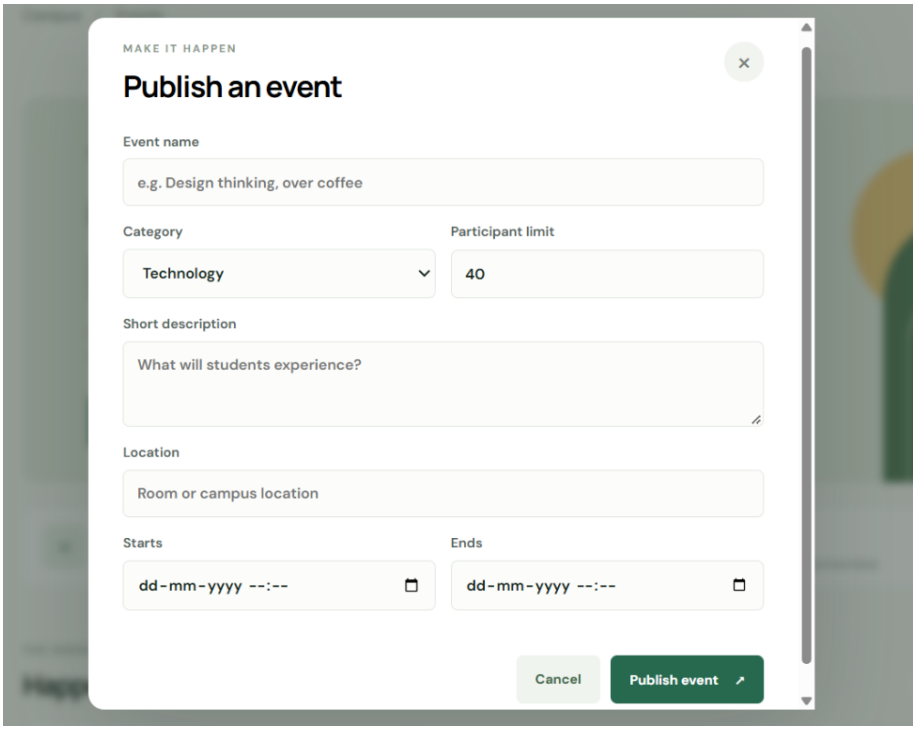


Figure 5. Event publishing form with title, category, participant limit, description, location, start and end inputs.

CHECK-IN

Student attendance page

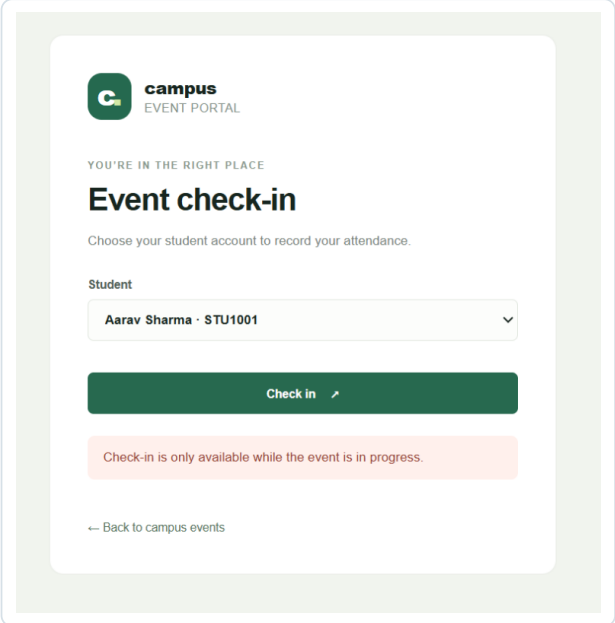


Figure 6. QR check-in page with student selection. The captured message states check-in is available only while the event is in progress.

CLOSING SUMMARY

Conclusion and future scope

CONCLUSION

Project summary and learning

The College Event Registration and Resource Booking Portal applies relational database principles to common campus-event workflows. Its schema separates student, event and resource entities from registration and reservation relationships. Keys and constraints define valid links and prevent duplicate records. The Node.js/Express service uses parameterized MySQL operations, transactions and row locks for capacity and inventory-sensitive workflows. QR check-in ties an attendance record to an active registration, and certificate generation depends on recorded attendance.

The project brings together frontend development, REST-style API design, MySQL schema design, database integrity, SQL querying and input validation. The verified automated evidence is limited to five request-validation tests; this report does not claim that all application features have end-to-end automated coverage.

FUTURE SCOPE

Potential improvements beyond the current demo

AREA	POSSIBLE IMPROVEMENT
Authentication & roles	Introduce secure student and organizer accounts, authorization and audit trails.
Event lifecycle	Add event updates/cancellation policies, organizer dashboards and controlled publishing permissions.
Notifications	Offer reminders and calendar export for registered students.
Testing & reliability	Add integration and concurrency tests for registrations, waitlists, reservations, check-in and certificates.
Reporting	Provide attendance summaries and resource-utilization analytics for authorized organizers.
Deployment	Configure production secrets, database backups, monitoring and a deployment environment.

IMPLEMENTED VS. FUTURE

The items above are recommendations only. Production authentication/authorization and reporting dashboards are not claimed as implemented in the supplied project.

No GitHub repository or live deployment URL was supplied; neither is asserted here.

SOURCES

References and project materials

OFFICIAL TECHNOLOGY DOCUMENTATION

Documentation for the stack used

TECHNOLOGY	DOCUMENTATION	URL
Node.js	Node.js API documentation	https://nodejs.org/docs/latest/api/
Express	Express 4.x API reference	https://expressjs.com/en/4x/api.html
MySQL	MySQL 8.0 Reference Manual	https://dev.mysql.com/doc/refman/8.0/en/
mysql2	mysql2 project documentation and repository	https://github.com/sidorares/node-mysql2
QRCode	node-qrcode project documentation and repository	https://github.com/soldair/node-qrcode
PDFKit	PDFKit documentation	https://pdfkit.org/docs/
HTML	MDN HTML documentation	https://developer.mozilla.org/en-US/docs/Web/HTML
CSS	MDN CSS documentation	https://developer.mozilla.org/en-US/docs/Web/CSS
JavaScript	MDN JavaScript documentation	https://developer.mozilla.org/en-US/docs/Web/JavaScript

PROJECT EVIDENCE

Files and supplied materials inspected

MATERIAL	USE IN THIS REPORT
Database definition	database/schema.sql — table structures, keys, constraints and indexes.
Sample records	database/seed.sql — example students, resources, events, registrations and attendance.
Application implementation	src/app.js and src/validation.js — API behavior, SQL workflows and request validation.
Dependencies and tests	package.json and test/validation.test.js — declared packages, scripts and automated test cases.
Academic formatting reference	Supplied rddbmsrpt.pdf — institute, university, department, academic year and semester details.
Application screenshots	Six supplied browser screenshots — reproduced on report pages 13–16.

Official documentation links are provided as references; no live application or repository URL is available.